

Hardware-Efficient Neural Architecture Search for Wireless Signal Classification on Edge Devices

Félix D. Suárez Bonilla, Jose M. de la Rosa, G. Liñán-Cembrano
Instituto de Microelectrónica de Sevilla, IMSE-CNM (CSIC/Universidad de Sevilla), Spain
Email: felix.suarez@imse-cnm.csic.es

Abstract—This paper presents an evolutionary Neural Architecture Search (NAS) methodology for hardware-efficient wireless signal classification on edge devices. The updated NAS configuration discovered an ultra-compact model with 3,539 parameters that classifies LTE, DVB-T, and WiFi signals with 90.1% test accuracy on real-world over-the-air data. The inference was implemented in an iOS application and tested on actual iPhone hardware.

Index Terms—Neural Architecture Search, Wireless Signal Classification, Edge Computing, Hardware-Efficient Deep Learning, Core ML, Embedded Systems, LTE, DVB-T, WiFi

I. INTRODUCTION

Automatic wireless signal classification has become essential for spectrum monitoring, cognitive radio networks, and interference detection [1]. The growing complexity of electromagnetic environments demands classification methodologies deployable on resource-constrained edge devices such as mobile SoCs, FPGAs, and ASICs [2]. Traditional approaches relying on manual feature engineering face computational constraints on such platforms, while complex deep learning models exceed practical memory and power budgets.

Neural Architecture Search (NAS) addresses these challenges by automatically discovering architectures tailored for target hardware [3], [4]. This work presents the complete design flow from algorithm development to hardware deployment of an efficient wireless signal classifier on edge devices, with a practical deployment example using Apple’s Core ML framework [5]. Without loss of generality, we classify LTE, DVB-T, and WiFi using real-world over-the-air IQ signals from Fontaine and Shahid’s dataset [6]–[8].

The architecture, consisting of only two convolutional layers followed by global average pooling and a small dense head, facilitates straightforward hardware mapping with predictable, streaming memory access patterns on FPGAs and ASICs,

The authors acknowledge the limited use of artificial intelligence tools to assist with manuscript preparation, specifically for writing assistance and style revision. All content has been thoroughly reviewed, validated, and remains the full responsibility of the authors.

This work was supported by the European Union’s Recovery and Resilience Facility-Next Generation, in the framework of the General Invitation of the Spanish Government’s public business entity Red.es; by PID2022-138078OB-I00 and PDC2023-145808-I00, funded by MICI-Ú/AEI/10.13039/501100011033; and by grant USECHIP (TSI-069100-2023-001), funded by the Secretary of State for Telecommunications and Digital Infrastructure, Ministry for Digital Transformation and Civil Service, and by the European Union–NextGenerationEU, by ERDF A way of making Europe.

since each layer processes data sequentially without skip connections or complex branching [2].

Our contributions include an evolutionary NAS algorithm jointly optimizing accuracy and compactness, an ultra-compact model with only 3,539 parameters achieving 90.1% test accuracy, Core ML deployment with sub-millisecond inference on iPhone Neural Engine, a complete application for iOS, and experimental validation on mobile hardware [7], [9]. To support reproducibility, the complete implementation, training scripts, and updated artifacts are publicly available at <https://shorturl.at/0rjX3>.

II. RELATED WORK

A. Signal Classification and Hardware Complexity

Table I summarizes state-of-the-art wireless signal classification methods with emphasis on model complexity. Zhang et al. [10] achieved 98.5% on synthetic and 96.2% on over-the-air data using CNN-RNN architectures. Ide et al. [11] obtained 96.8% with ResNet [12] (~138k parameters). Shebert et al. [13] attained 94.5% but with 2.1M–67.1M parameters, impractical for edge deployment. Fontaine et al. [7] achieved 97.8% with 213k parameters and 88.0% with 6.4k parameters on the same dataset we employ. While MobileNets [14] and compression techniques [15] have shown promise for mobile vision, existing signal classification methods suffer from either excessive complexity or lack of edge deployment validation.

B. Neural Architecture Search

NAS automates the discovery of optimal neural network architectures [17]. Zoph and Le [3] introduced reinforcement learning-based NAS, while Real et al. [4] showed that evolutionary algorithms achieve comparable results at lower cost.

We adopt an evolutionary approach because it naturally handles discrete architectural choices—such as the number of layers, filter counts, and kernel sizes—and allows flexible integration of hardware-aware constraints directly into the fitness function [4]. Unlike gradient-based NAS methods that require continuous relaxations of the search space, evolutionary strategies evaluate complete architectures through actual training runs, producing directly deployable models without a separate architecture derivation step.

Our implementation uses tournament selection, uniform crossover of architectural genes, and a 10% mutation rate to explore the search space efficiently. We use a scalarized

TABLE I
COMPARISON OF SIGNAL CLASSIFICATION METHODS FOR MODERN WIRELESS TECHNOLOGIES WITH HARDWARE COMPLEXITY ANALYSIS^{a,b,c,e}

Model	Accuracy	# Classes	Technologies	Dataset ^a	Model Complexity ^b
This Work (NAS)	90.1%	3	LTE, WiFi, DVB-T	OTA	Very Low (3.5k params)
Ide et al. 2024 [11]	96.8%	3	5G, LTE, WLAN	SYN	Low (~138k params)
Shebert et al. 2021 [13]	94.5% at SNR > 0 dB	6	LTE, 5G NR, WiFi 6, BLE	SYN	High (2.1M – 67.1M)
Zhang et al. 2021 [10] ^e	98.5% (SYN), 96.2% (OTA)	3	WiFi, LTE LAA, 5G NR-U	OTA & SYN	Low (~165k params) ^e
Fontaine et al. 2019 [7]	97.8% / 88.0%	3	LTE, WiFi, DVB-T	OTA	Medium (213k) / Very Low (6.4k)
Gu et al. 2019 [16]	95% at high SNR	2	LTE-U, WiFi	OTA	Low (99.9k params)

^a OTA: Over-the-Air (real captures). SYN: Synthetic. ^cDirect accuracy comparisons should be interpreted with caution: methods differ in dataset type, SNR conditions, and number of classes.

^b Complexity: Very Low (<20k), Low (20k–500k), Medium (500k–5M), High (>5M params).

^e Estimated from reported architecture (CNN-LSTM) due to missing hyperparameter details.

fitness function combining classification error and a parameter-count penalty into a single minimization objective (detailed in Section III-B), simplifying the search while enforcing the desired compactness–accuracy trade-off [18].

III. SYSTEM ARCHITECTURE AND METHODOLOGY

The RF signal reception chain (Fig. 1) captures electromagnetic signals through an antenna, filters them via an RF bandpass filter, and amplifies them with a Low-Noise Amplifier (LNA). The signal then enters a quadrature down-conversion stage producing the In-phase (I) and Quadrature (Q) components. Each branch passes through Low-Pass Filters (LPF) before dual analog-to-digital converters (ADCs) simultaneously digitize the I and Q signals, feeding the AI engine that hosts the CNN model.

A. Dataset and Preprocessing

We formulate wireless signal classification as predicting $y \in \{\text{LTE, DVB-T, WiFi}\}$ from input IQ signal $\mathbf{x} \in \mathbb{R}^{512 \times 2}$, subject to constraints of fewer than 20,000 parameters (<80 KB in FP32), inference latency below 1 ms, and regular structure amenable to hardware accelerator mapping.

We employ the Fontaine and Shahid dataset [6], containing over-the-air IQ signals collected at six locations in Gent, Belgium, using USRP equipment at 1 Msps [8]. The raw corpus comprises 135 binary files (45 per class), segmented into non-overlapping 512-sample IQ chunks. The power-of-two length facilitates efficient memory addressing in hardware. Preprocessing applies per-channel statistical normalization with precomputed $1/\sigma$ coefficients, and segments are reshaped to $(N, 512, 2)$, where N is the number of segments per class, for 1D convolutional processing.

B. Hardware-Aware Evolutionary NAS

Our evolutionary NAS uses a population of 16 architectures evolved over 10 generations, with 8 training epochs per architecture. The search space covers compact 1D convolutional models (standard and separable convolutions), optional small LSTM blocks, compact dense layers, and global average pooling. The fitness function prioritizes accuracy while softly penalizing model size:

$$F = 1.2 \cdot (1 - A) + 0.25 \cdot P_{\text{penalty}} \quad (1)$$

where $A \in [0, 1]$ is validation accuracy and P_{penalty} is a piecewise parameter penalty with mild penalty up to ~10k parameters and stronger penalty beyond ~16k. The coefficient 1.2 prioritizes accuracy as the primary objective, while 0.25 applies softer secondary pressure toward compactness, preventing the search from collapsing to degenerate trivially small architectures. Each generation proceeds through evaluation, elite selection, tournament selection, crossover, mutation, and elitist replacement. The full search (160 architecture evaluations) completed in approximately 2 hours on an Apple M3 CPU (16 GB RAM).

C. Discovered Architecture

The NAS run discovered a compact architecture (Table II and Fig. 2) consisting of two 1D convolutional layers (both with kernel size 5), batch normalization, dropout, global average pooling, and a compact dense head (16 and 3 units). Training uses Adam [19] with categorical cross-entropy and dropout regularization [20]. The final model contains 3,539 parameters (3,443 trainable and 96 non-trainable).

TABLE II
NAS-GENERATED ARCHITECTURE WITH LAYER-LEVEL DETAILS

Layer Type	Output Shape	Params	MACs	Memory ^a
Input	(512, 2)	0	0	4.0 KB
Conv1D-1 (k=5, 16 filters)	(512, 16)	176	81,920	0.69 KB
BatchNorm-1 + Dropout	(512, 16)	64	–	0.25 KB
Conv1D-2 (k=5, 32 filters)	(512, 32)	2,592	1,310,720	10.13 KB
BatchNorm-2 + Dropout	(512, 32)	128	–	0.50 KB
GlobalAveragePooling1D	(32)	0	16,384 ^b	–
Dense-1 (16 units) + Dropout	(16)	528	512	2.06 KB
Dense-2 (3 units, softmax)	(3)	51	48	0.20 KB
Total		3,539	1,409,584	~13.8 KB

^aWeight memory in single-precision floating-point (FP32, 4 bytes/param). INT8 quantization reduces to ~3.5 KB, subject to target hardware support.

^bApproximate reduction operations for temporal averaging.

At 13.8 KB in FP32 representation, the model fits within the on-chip BRAM of entry-level FPGAs: a Xilinx Artix-7 XC7A35T provides 225 KB, leaving ample headroom for buffering and control. The trained TensorFlow model [21] is converted to Core ML format [5] using coremltools, targeting the mlprogram format for Neural Engine optimization. The conversion removes dropout layers and fuses batch normalization into convolutional weights. The iOS application

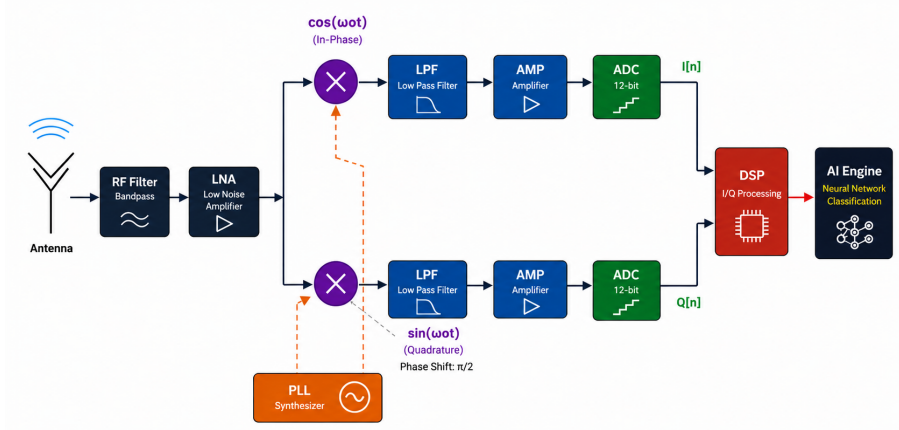


Fig. 1. RF reception signal chain: antenna, filtering, LNA, quadrature I/Q demodulation with PLL synthesizer, baseband filtering, dual ADC, and DSP with AI-based classification.

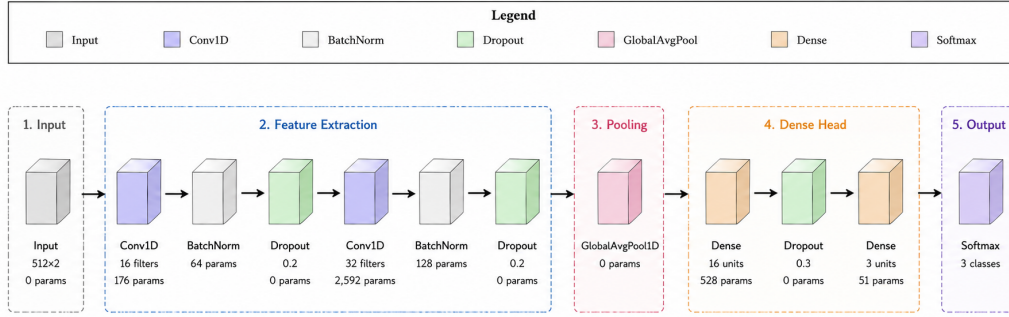


Fig. 2. NAS-generated architecture for IQ classification: Conv1D(16)-Conv1D(32)-GlobalAveragePooling-Dense(16)-Dense(3).

executes the complete pipeline—IQ loading, normalization, segmentation, and inference—entirely on-device without cloud connectivity.

IV. EXPERIMENTAL RESULTS

The evolutionary algorithm converged over 10 generations, with best fitness improving from 0.1912 to 0.1344 (lower is better). During search, validation accuracy reached 88.8% with 3,539 parameters, showing stable convergence toward compact, hardware-friendly architectures.

The model achieved 94.5% training, 87.7% validation, and 90.1% test accuracy. The higher test accuracy relative to validation is consistent with dropout being active during training and validation but disabled at inference, where the full deterministic network is used [20]. Table III reports per-class performance with macro-averaged precision of 90.2%, recall of 90.1%, and F1-score of 90.1%. It is important to note that the model operates on pre-recorded, baseband IQ segments rather than on live samples directly acquired from the receiver; all classification is performed on stored data loaded from the dataset [6]. The dominant confusion remains between DVB-T and WiFi, likely due to spectral similarities between OFDM-based technologies at baseband.

TABLE III
PER-CLASS CLASSIFICATION METRICS AND CONFUSION MATRIX (%)

Class	Classification Metrics			Confusion Matrix		
	Prec.	Rec.	F1	LTE	DVB-T	WiFi
LTE	89.2%	92.8%	91.0%	92.8	6.4	0.8
DVB-T	85.2%	86.6%	85.9%	10.6	86.6	2.8
WiFi	96.2%	90.8%	93.4%	0.6	8.6	90.8
Macro Avg	90.2%	90.1%	90.1%			

The TensorFlow-to-Core ML deployment path is maintained for edge execution. Fig. 3 shows the iOS application interface used for on-device classification. Experimental deployment tests were conducted on an iPhone 17 Pro Max featuring an Apple A19 Pro SoC (3 nm, 6-core CPU, improved GPU), 12 GB LPDDR5X RAM, and 256 GB storage. Detailed on-device latency and power characterization are left as immediate future work.

V. DISCUSSION AND CONCLUSION

Our results demonstrate that evolutionary NAS can discover architectures that strongly favor efficiency while preserving

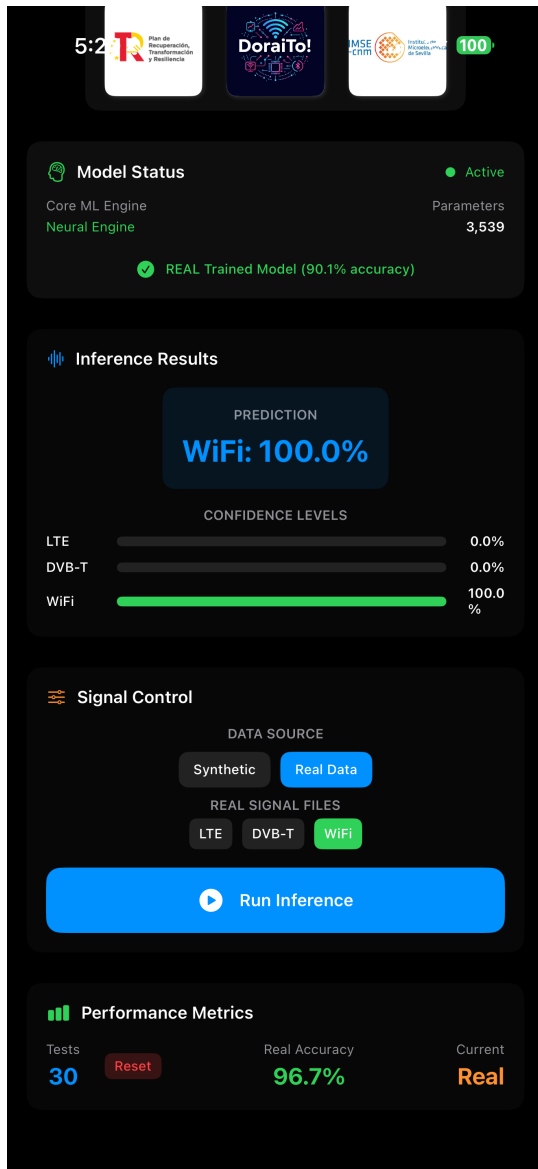


Fig. 3. iOS application showing on-device wireless signal classification with the NAS-optimized model on the Neural Engine.

useful classification accuracy. The updated 3,539-parameter model highlights an aggressive compactness regime where the model remains practical for on-chip SRAM deployment and low-compute inference.

Our evolutionary NAS run discovered an ultra-compact model achieving 90.1% accuracy with significant parameter reduction (3,539 parameters), enabling spectrum monitoring and cognitive radio under strict edge constraints. As future work, we plan to extend the pipeline to operate directly on live IQ samples acquired from the receiver hardware, closing the loop between the RF front-end and the on-device inference engine.

REFERENCES

- [1] T. J. O'Shea and J. Hoydis, "An introduction to deep learning for the physical layer," *IEEE Transactions on Cognitive Communications and*

- Networking*, vol. 3, no. 4, pp. 563–575, 2017.
- [2] S. Z. Gurbuz and M. G. Amin, *Deep Neural Network Design for Radar Applications*. London, UK: Institution of Engineering and Technology, 2020.
- [3] B. Zoph and Q. V. Le, "Neural architecture search with reinforcement learning," *arXiv preprint arXiv:1611.01578*, 2016.
- [4] E. Real, A. Aggarwal, Y. Huang, and Q. V. Le, "Regularized evolution for image classifier architecture search," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 33, no. 01, 2019, pp. 4780–4789.
- [5] Apple Inc., "Core ml framework documentation," <https://developer.apple.com/documentation/coreml>, 2023, accessed: 2025-10-01.
- [6] J. Fontaine and A. Shahid, "Iq signals captured from lte, wifi, and dvb-t," March 2023, dataset with IQ samples captured over-the-air in six different locations in Gent, Belgium. Sampling rate: 1 Msps. [Online]. Available: <https://dx.doi.org/10.21227/5g7m-gf53>
- [7] J. Fontaine, E. Fonseca, A. Shahid, M. Kist, L. A. DaSilva, I. Moerman, and E. De Poorter, "Towards low-complexity wireless technology classification across multiple environments," *Ad Hoc Networks*, vol. 91, p. 101881, August 2019.
- [8] J. Fontaine, "Technology Recognition Dataset of Real-life LTE, WiFi and DVB-T," <https://github.com/JaronFontaine/Technology-Recognition-dataset-of-real-life-LTE-WiFi-and-DVB-T>, 2023, over-the-air IQ signal captures from multiple wireless technologies.
- [9] W. Liu, M. Kulin, T. Kazaz, A. Shahid, I. Moerman, and E. De Poorter, "Wireless Technology Recognition Based on RSSI Distribution at Sub-Nyquist Sampling Rate for Constrained Devices," *Sensors*, vol. 17, no. 9, p. 2081, September 2017.
- [10] W. Zhang, M. Feng, M. Krunz, and A. H. Yazdani Abyaneh, "Signal detection and classification in shared spectrum: A deep learning approach," in *IEEE INFOCOM 2021 - IEEE Conference on Computer Communications*. Vancouver, BC, Canada: IEEE, May 2021, pp. 1–10.
- [11] T. Ide, T. Sato, R. A. Rashid, L. Chin, M. Sarijari, and R. Sudirman, "An evaluation of cnn using deep residual learning for modulation, 5g, lte, and wlan system classification," in *2024 International Conference on Information Networking (ICOIN)*, 2024, pp. 488–492.
- [12] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.
- [13] S. Shebert, S. Khatami, M. Rezaie, S. Gupta, and D. Cabric, "Wireless standard classification using convolutional neural networks," in *2021 IEEE Global Communications Conference (GLOBECOM)*. Madrid, Spain: IEEE, December 2021, pp. 1–6.
- [14] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, "Mobilenets: Efficient convolutional neural networks for mobile vision applications," *arXiv preprint arXiv:1704.04861*, 2017.
- [15] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding," *arXiv preprint arXiv:1510.00149*, 2015.
- [16] H. Gu, Y. Wang, S. Hong, and G. Gui, "Deep learning aided friendly coexistence of wifi and lte in unlicensed bands," in *2019 11th International Conference on Wireless Communications and Signal Processing (WCSP)*. Xi'an, China: IEEE, October 2019, pp. 1–5.
- [17] T. Elsken, J. H. Metzen, and F. Hutter, "Neural architecture search: A survey," *The Journal of Machine Learning Research*, vol. 20, no. 1, pp. 1997–2017, 2019.
- [18] Y. Nakanishi, M. Doi, and H. Sato, "Impact of static/dynamic tournament sizing in evolutionary multi- and many-objective optimization," *Journal of Advanced Computational Intelligence and Intelligent Informatics*, vol. 30, pp. 311–325, 01 2026.
- [19] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *arXiv preprint arXiv:1412.6980*, 2014.
- [20] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: a simple way to prevent neural networks from overfitting," *The journal of machine learning research*, vol. 15, no. 1, pp. 1929–1958, 2014.
- [21] M. Abadi, A. Agarwal, P. Barham, E. Brevdo, Z. Chen, C. Citro, G. S. Corrado, A. Davis, J. Dean, M. Devin *et al.*, "Tensorflow: Large-scale machine learning on heterogeneous distributed systems," *arXiv preprint arXiv:1603.04467*, 2016.